



in collaboration with



Assignment Title:

Lab 3:

Investigating Process Lifecycles and OS Interaction.

Course name:

*BSc. (Hons) Computer Science with AI, Software College of IT and E-Commerce,
Coventry University*

CourseWork Type:

Individual

Module Name:

ST5003CMD Operating Systems, Security, and Networks

Intake: *September*

Submitted By:

CU ID: 16536799

College ID: 250073

Name: Ashish Shrestha(A)

Submitted To:

Rupak Rajbanshi

TABLE OF CONTENTS:

I. INTRODUCTION	1
II. TASK 1: THE LONG-RUNNING PROCESS	2
A. C Source Code	2
B. Terminal Execution & Verification	3
III. TASK 2: PROCESS IDENTITY (PID AND PPID)	4
A. C Source Code	4
B. Terminal Execution & Verification	5
IV. TASK 3: EXIT CODES AND OS FEEDBACK	6
A. C Source Code	6
B. Terminal Execution & Verification	7
C. Scenario A: Positive Input	7
D. Scenario B: Negative Input	8
V. TASK 4: STANDARD I/O STREAMS	9
A. C Source Code	9
B. Terminal Execution	10
VI. TASK 5: CONDITIONAL EXECUTION AND TERMINATION	11
A. C Source Code	11
B. Terminal Execution & Verification	12
VII. CONCLUSION	14

LIST OF FIGURES:

Figure 1: Command to print “I am finished” in C for sleep	2
Figure 2: Command to print “I am finished” in C	2
Figure 3: Create object for sleep_process.c file	3
Figure 4: Use & to run the process in the background	3
Figure 5: Monitor the process using ps aux grep	3
Figure 6: Command to check Process Identity (PID) and Parent Process ID (PPID)	4
Figure 7: C source code for PID and PPID	5
Figure 8: C source code for PID and PPID	5
Figure 9: Command to check PID, PPID, and CMD	5
Figure 10: C source code that returns 0 for success and 1 for failure	6
Figure 11: Command that returns success for positive input and failure for negative input	7
Figure 12: Command that creates the executable for the C program	7
Figure 13: Command that gives success for positive user input	7
Figure 14: echo \$? command showing the exit status	8
Figure 15: Command that gives failure for negative user input	8
Figure 16: echo \$? command showing exit status 1 for failure	8
Figure 17: C source code for standard input and standard output streams	9
Figure 18: Command demonstrating standard input and standard output	9
Figure 19: Command that creates the executable for standard_I_O_stream.c	10
Figure 20: Command that takes user input and displays it as output	10
Figure 21: C source code for conditional execution and termination	11
Figure 22: Command demonstrating conditional execution and termination	11
Figure 23: Command that creates the executable for execution_and_termination.c ...	12
Figure 24: Command that continues when user input is 1	12
Figure 25: Command that exits when user input is 0	13
Figure 26: echo \$? command showing exit status 0	13
Figure 27: echo \$? command showing exit status 1	13

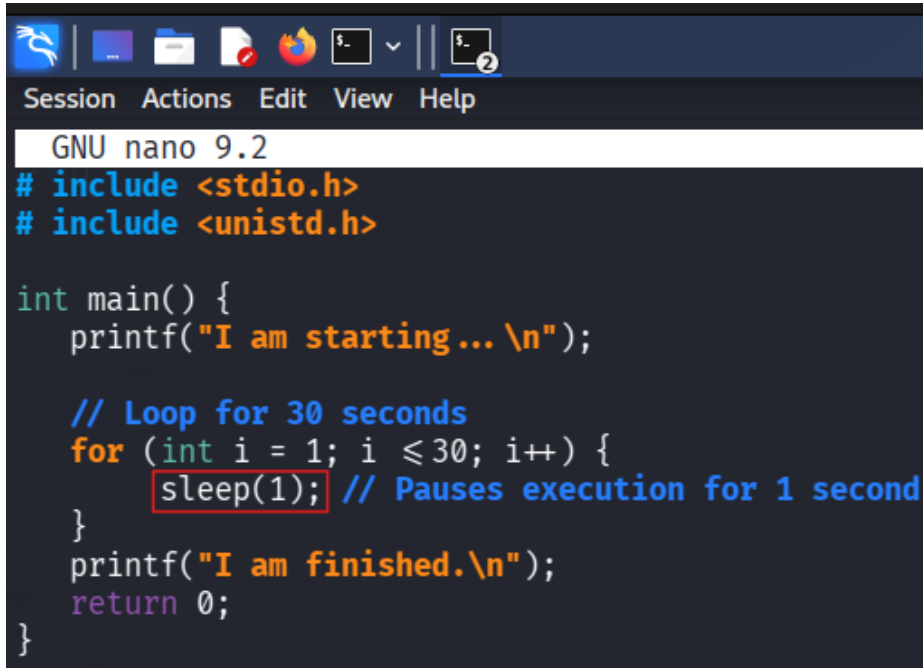
I. INTRODUCTION:

In Lecture 3, we learned that an OS manages processes and resources. In this lab, we bring that theory to life. Instead of just writing code that prints text, we will write code that interacts with the Linux kernel to query its own process identity, control its execution time, and report its success or failure back to the OS shell.

II. TASK 1: THE LONG-RUNNING PROCESS:

A. C Source Code:

We create a program that runs for 30 seconds using a loop and sleep().

A screenshot of a terminal window with a dark background. The window title bar shows icons for a terminal, a file manager, and a web browser. Below the title bar is a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The main area shows the GNU nano 9.2 editor with C code. The code includes <stdio.h> and <unistd.h>. The main function prints 'I am starting... \n', then enters a for loop from i=1 to i=30. Inside the loop, there is a call to sleep(1); which is highlighted with a red box. A comment next to it says '// Pauses execution for 1 second'. After the loop, it prints 'I am finished.\n' and returns 0.

```
GNU nano 9.2
# include <stdio.h>
# include <unistd.h>

int main() {
    printf("I am starting... \n");

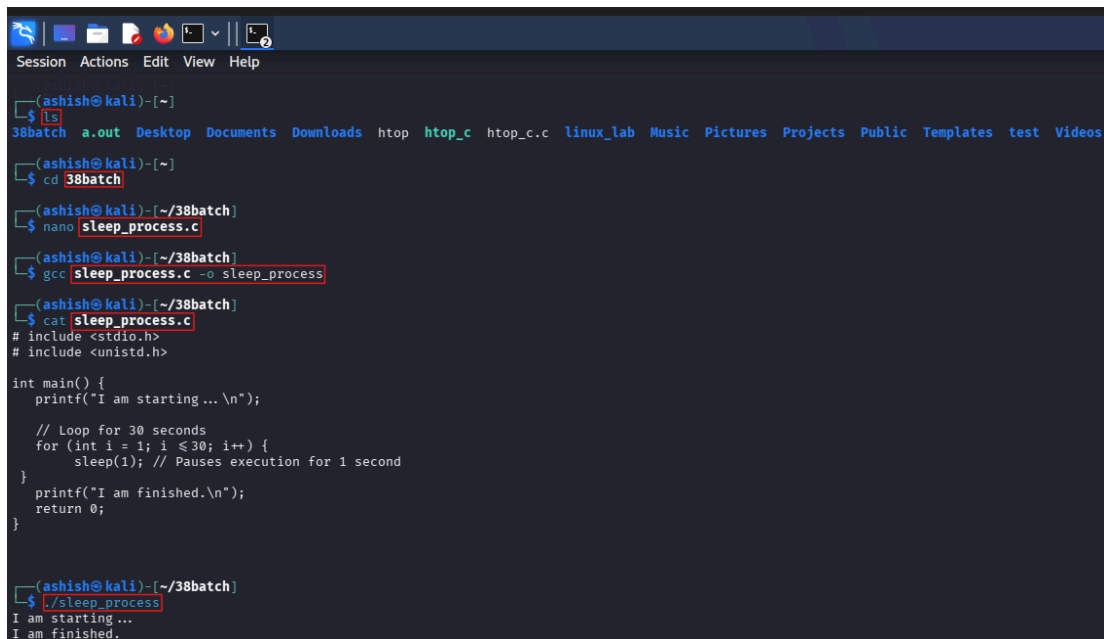
    // Loop for 30 seconds
    for (int i = 1; i <= 30; i++) {
        sleep(1); // Pauses execution for 1 second
    }
    printf("I am finished.\n");
    return 0;
}
```

Fig1: Command to print "I am finished" in C for sleep.

OS Insight:

In enterprise servers, processes run for days or months. We must know how to monitor them while they are active.

1. File Operation: This command returns output “I am finished.” for users in the C program.



```
(ashish@kali)~$ ls
38batch  a.out  Desktop  Documents  Downloads  htop  htop_c  htop_c.c  linux_lab  Music  Pictures  Projects  Public  Templates  test  Videos

(ashish@kali)~$ cd 38batch

(ashish@kali)~/38batch$ nano sleep_process.c

(ashish@kali)~/38batch$ gcc sleep_process.c -o sleep_process

(ashish@kali)~/38batch$ cat sleep_process.c
# include <stdio.h>
# include <unistd.h>

int main() {
    printf("I am starting ... \n");

    // Loop for 30 seconds
    for (int i = 1; i <= 30; i++) {
        sleep(1); // Pauses execution for 1 second
    }

    printf("I am finished.\n");
    return 0;
}

(ashish@kali)~/38batch$ ./sleep_process
I am starting ...
I am finished.
```

Fig2: Command to print “I am finished.” in C.

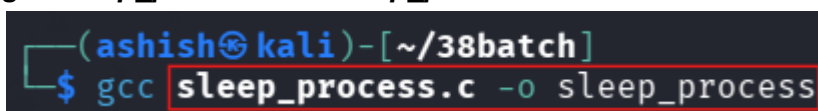
B. Terminal Execution & Verification:

To make this interesting, we run the program in the background using & so we can use a second terminal to inspect it while it's alive.

I. compile the code:

➤ Command Create object for sleep_proces.c file:

gcc sleep_process.c -o sleep_process



```
(ashish@kali)~/38batch$ gcc sleep_process.c -o sleep_process
```

Fig3: Create object for sleep_process.c file.

II. Run in background (&):

➤ `./sleep_process &` to check process id:

`./sleep_process &`

```
(ashish@kali)-[~/38batch]
$ ./sleep_process &
[1] 23851

I am starting ...
(ashish@kali)-[~/38batch]
$ I am finished.

[1] + done      ./sleep_process
```

Fig4: use & to inspect output.

➤ `ps aux | grep` command to to check current process:

`ps aux | grep sleep_process`

```
(ashish@kali)-[~/38batch]
$ ps aux | grep sleep_process
ashish  29572  0.0  0.0  6556  2384 pts/0    S+   02:56   0:00 grep --color=auto sleep_process
```

Fig5: Monitor the process.

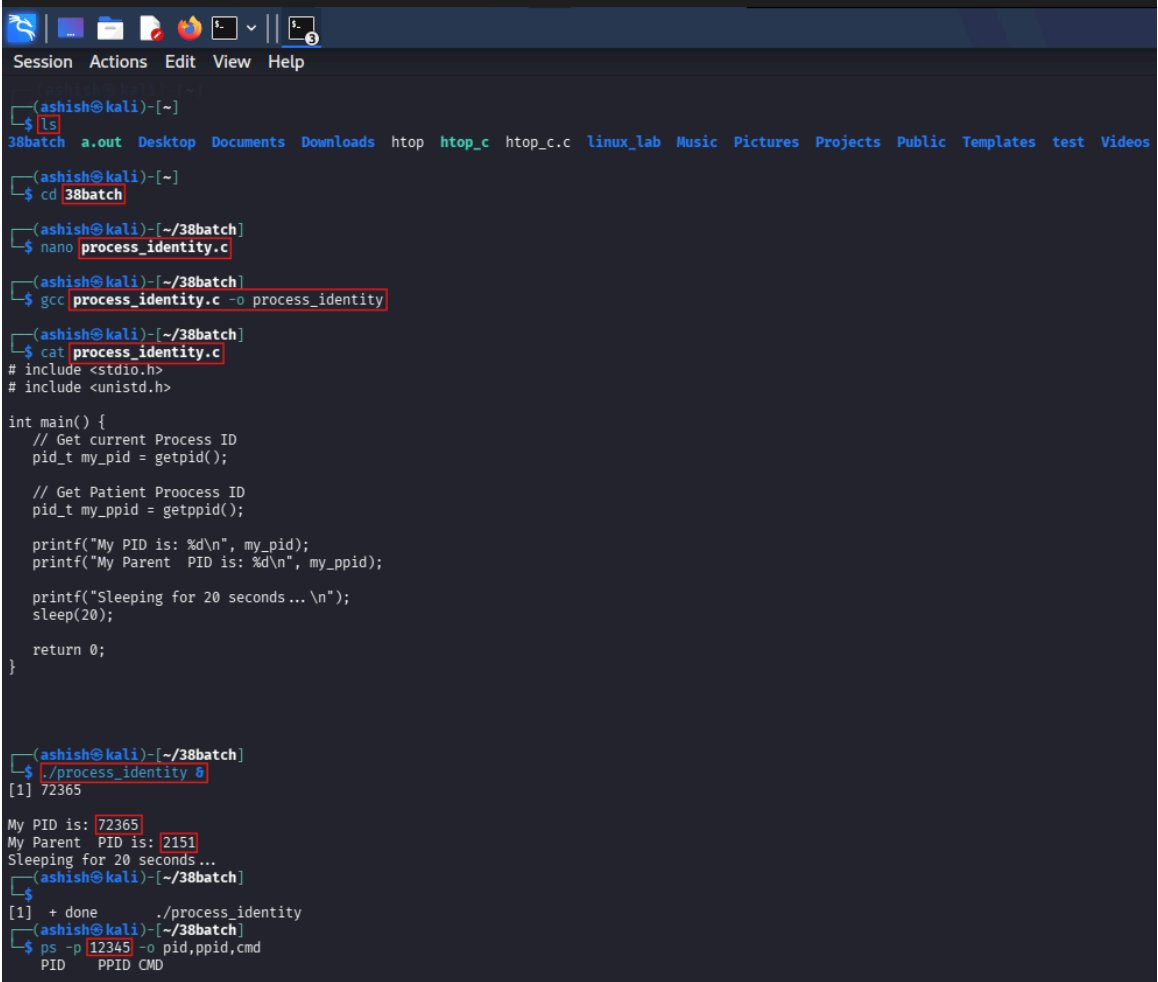
III. TASK 2: PROCESS IDENTITY (PID AND PPID)

OS Insight: Every process in Linux has a unique Process ID (PID).

The process that created it is the Parent Process ID (PPID), usually your terminal shell.

III. TASK 2: PROCESS IDENTITY (PID AND PPID):

2.File Operation: This command returns output “I am finished.” for users in the C program.



```
(ashish@kali)-[~]
$ ls
38batch  a.out  Desktop  Documents  Downloads  htop  htop_c  htop_c.c  linux_lab  Music  Pictures  Projects  Public  Templates  test  Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano process_identity.c

(ashish@kali)-[~/38batch]
$ gcc process_identity.c -o process_identity

(ashish@kali)-[~/38batch]
$ cat process_identity.c
# include <stdio.h>
# include <unistd.h>

int main() {
    // Get current Process ID
    pid_t my_pid = getpid();

    // Get Parent Process ID
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);

    return 0;
}

(ashish@kali)-[~/38batch]
$ ./process_identity
[1] 72365

My PID is: 72365
My Parent PID is: 2151
Sleeping for 20 seconds...
(ashish@kali)-[~/38batch]
$
[1] + done ./process_identity
(ashish@kali)-[~/38batch]
$ ps -p 12345 -o pid,ppid,cmd
PID    PPID CMD
```

Fig6: Command to check process Process Identity (PID) and Parent Process Identity(PPID).

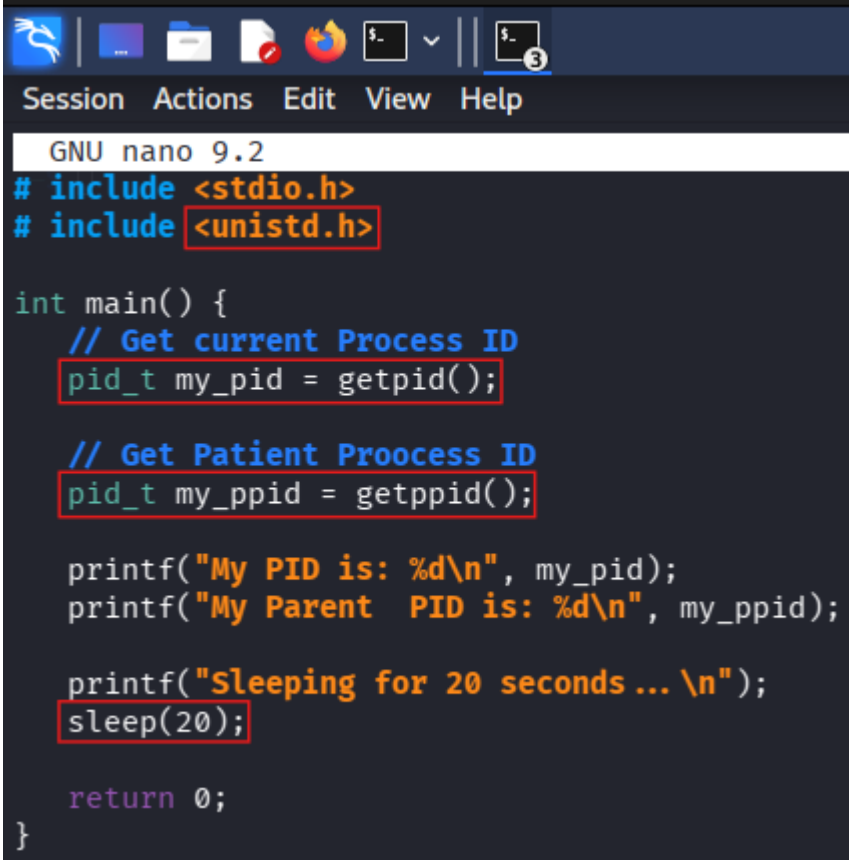
OS Insight: Every process in Linux has a unique Process ID (PID).

The process that created it is the Parent Process ID (PPID), usually your terminal shell.

II. TASK 1: THE LONG-RUNNING PROCESS:

A. C Source Code:

We created a program that created a unique ID (PID). The process that created it is the Parent Process ID (PPID).

A screenshot of a GNU nano 9.2 text editor window. The window has a dark blue title bar with icons for file operations and a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The code is written in a dark-themed editor with syntax highlighting. The code includes headers for stdio.h and unistd.h, defines a main function that uses getpid() and getppid() to retrieve the current process ID and parent process ID respectively, prints these values, sleeps for 20 seconds, and then returns 0. Red boxes highlight the include statements, the variable declarations, and the function calls.

```
GNU nano 9.2
# include <stdio.h>
# include <unistd.h>

int main() {
    // Get current Process ID
    pid_t my_pid = getpid();

    // Get Patient Proocess ID
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds ... \n");
    sleep(20);

    return 0;
}
```

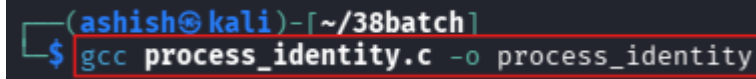
Fig7: C Source code for ID (PID) and Parent Process ID (PPID).

B. Terminal Execution & Verification:

I. Create Object for Process Identity Code(PID) and Parent Process Identity Code(PPID) using C:

➤ GCC command to create an object for process_identity.c file:

`gcc process_identity.c -o process_identity`



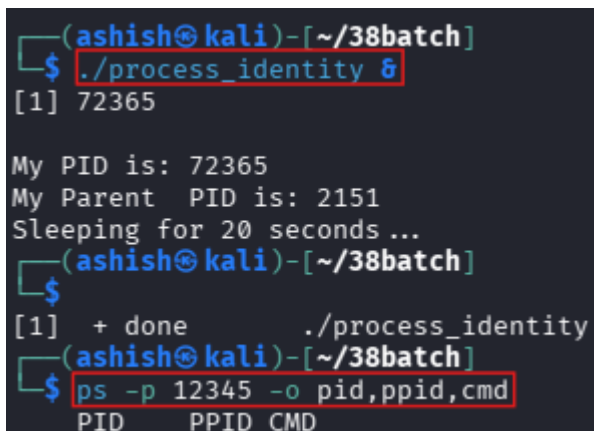
```
(ashish@kali)-[~/38batch]
$ gcc process_identity.c -o process_identity
```

Fig8: C Source code for ID (PID) and Parent Process ID (PPID).

Verify it using the OS ps command:

➤ GCC command to create an object for process_identity.c file:

`gcc process_identity.c -o process_identity`



```
(ashish@kali)-[~/38batch]
$ ./process_identity 8
[1] 72365

My PID is: 72365
My Parent PID is: 2151
Sleeping for 20 seconds ...
(ashish@kali)-[~/38batch]
$
[1] + done ./process_identity
(ashish@kali)-[~/38batch]
$ ps -p 12345 -o pid,ppid,cmd
PID PPID CMD
```

Fig9: Command to check PID, PPID, CMD.

IV. TASK 3: EXIT CODES AND OS FEEDBACK

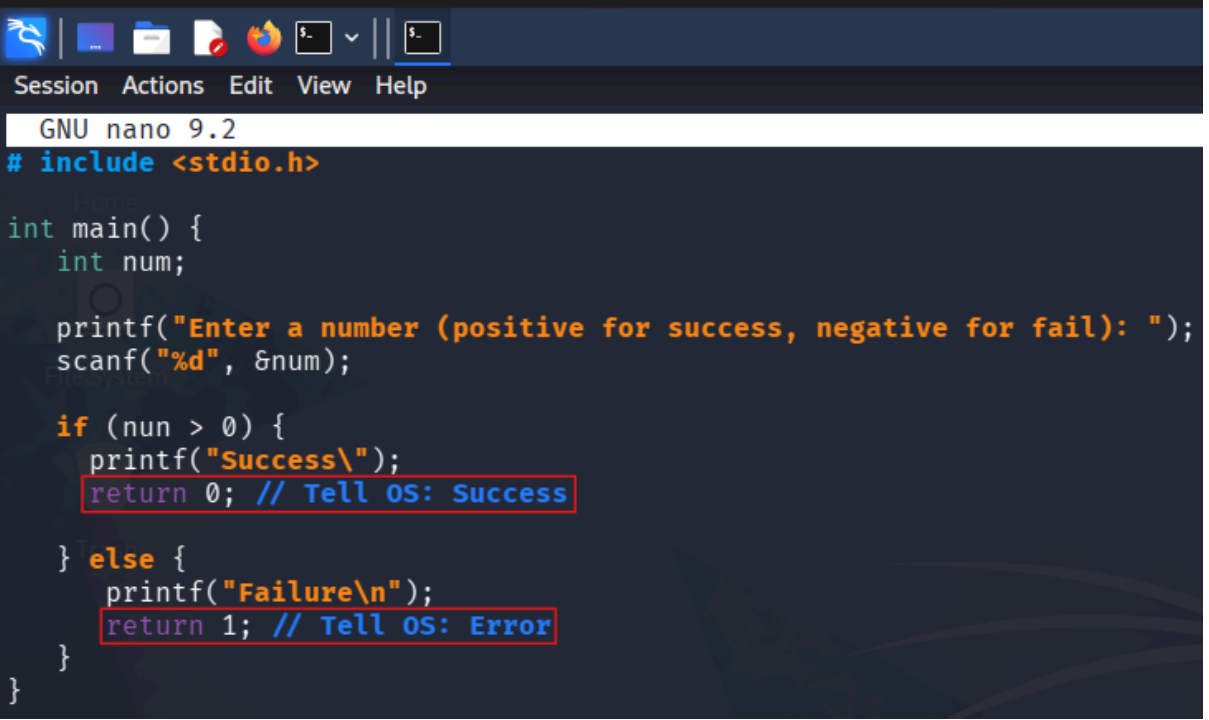
OS Insight: When a program finishes, it returns an integer to the OS.

0 means "Success", and any non-zero number (like 1) means "Error/Failure". The shell stores this in \$?.

II. TASK 1: THE LONG-RUNNING PROCESS:

A. C Source Code:

C Code program finishes, it returns as integer to the OS. 0 means "Success", and any non-zero number (like 1) means "Error/Failure", The shell stores this in \$?.

A screenshot of a code editor window titled 'GNU nano 9.2'. The editor contains C source code for a program that takes a number as input and returns an exit code. The code is as follows:

```
#include <stdio.h>

int main() {
    int num;

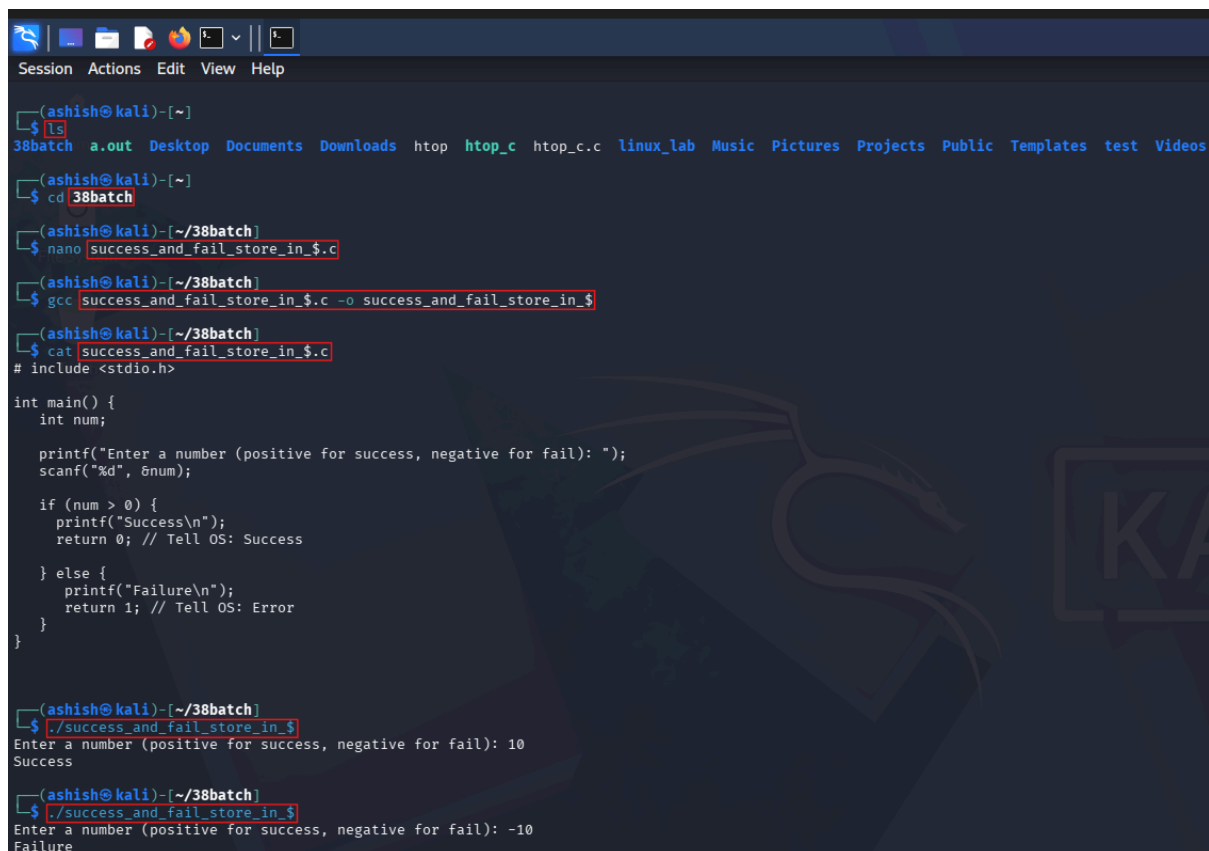
    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Success\n");
        return 0; // Tell OS: Success
    } else {
        printf("Failure\n");
        return 1; // Tell OS: Error
    }
}
```

The code uses color syntax highlighting: keywords like 'include', 'int', 'if', 'else', and 'return' are in blue; string literals are in orange; and comments are in green. The two 'return' statements are highlighted with red rectangular boxes.

Fig10: C source code that returns 0 for "Success " and 1 for "Failure.

3. File Operation: This command returns Success for positive input and fail for negative input values.



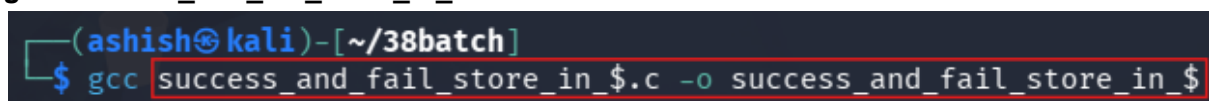
```
(ashish@kali)~  
$ ls  
38batch a.out Desktop Documents Downloads htop htop_c htop_c.c linux_lab Music Pictures Projects Public Templates test Videos  
(ashish@kali)~  
$ cd 38batch  
(ashish@kali)~/38batch  
$ nano success_and_fail_store_in_$.c  
(ashish@kali)~/38batch  
$ gcc success_and_fail_store_in_$.c -o success_and_fail_store_in_$.  
(ashish@kali)~/38batch  
$ cat success_and_fail_store_in_$.c  
# include <stdio.h>  
  
int main() {  
    int num;  
  
    printf("Enter a number (positive for success, negative for fail): ");  
    scanf("%d", &num);  
  
    if (num > 0) {  
        printf("Success\n");  
        return 0; // Tell OS: Success  
    } else {  
        printf("Failure\n");  
        return 1; // Tell OS: Error  
    }  
}  
  
(ashish@kali)~/38batch  
$ ./success_and_fail_store_in_$.  
Enter a number (positive for success, negative for fail): 10  
Success  
(ashish@kali)~/38batch  
$ ./success_and_fail_store_in_$.  
Enter a number (positive for success, negative for fail): -10  
Failure
```

Fig11: Command that returns output according to user input Success of positive input and negative for Failure.

B. Terminal Execution & Verification:

➤ GCC command to create object:

gcc success_and_fail_store_in_\$.c



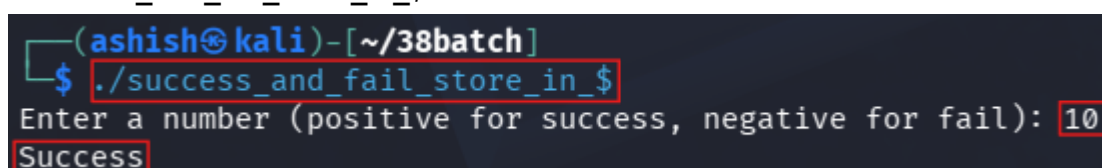
```
(ashish@kali)~/38batch  
$ gcc success_and_fail_store_in_$.c -o success_and_fail_store_in_$.
```

Fig12: Command that creates object of success_and_fail_store_in_\$.c C program File.

C. Scenario A: Positive Input:

➤ Command for positive output as Success:

./success_and_fail_store_in_\$.c



```
(ashish@kali)~/38batch  
$ ./success_and_fail_store_in_$.  
Enter a number (positive for success, negative for fail): 10  
Success
```

Fig13: Command that gives Success for positive user_input.

Scenario A: Positive Input:

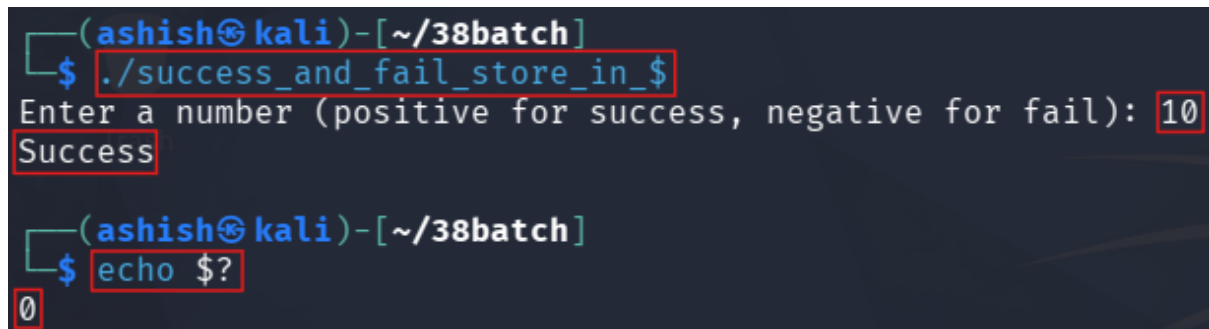
Enter: 10

Output: Success

➤ **echo \$? Command to check 0 as output for success:**

echo \$?

Output: 1

A terminal window with a dark background. The prompt is (ashish@kali)-[~/38batch]. The user enters ./success_and_fail_store_in_\$. The output is "Enter a number (positive for success, negative for fail): 10" followed by "Success". Then, the user enters echo \$?. The output is 0. Red boxes highlight the command ./success_and_fail_store_in_\$, the input 10, the output Success, the command echo \$?, and the output 0.

```
(ashish@kali)-[~/38batch]
$ ./success_and_fail_store_in_$
Enter a number (positive for success, negative for fail): 10
Success

(ashish@kali)-[~/38batch]
$ echo $?
0
```

Fig14: echo \$? Command which returns 0 for positive user_input for success .

Scenario B: Negative Input:

Enter: -10

Output: Failure

➤ **Command for negative output as Failure:**

./success_and_fail_store_in_\$.c

```
(ashish@kali)-[~/38batch]
$ ./success_and_fail_store_in_$.c
Enter a number (positive for success, negative for fail): -10
Failure
```

Fig15: Command that gives Failure for negative user_input.

➤ **echo \$? Command to check 1 as output for failure:**

echo \$?

Output: 1

```
(ashish@kali)-[~/38batch]
$ ./success_and_fail_store_in_$.c
Enter a number (positive for success, negative for fail): -10
Failure

(ashish@kali)-[~/38batch]
$ echo $?
1
```

Fig16: echo \$? Command which returns 1 for negative user_input for Failure.

V. TASK 4: STANDARD I/O STREAMS

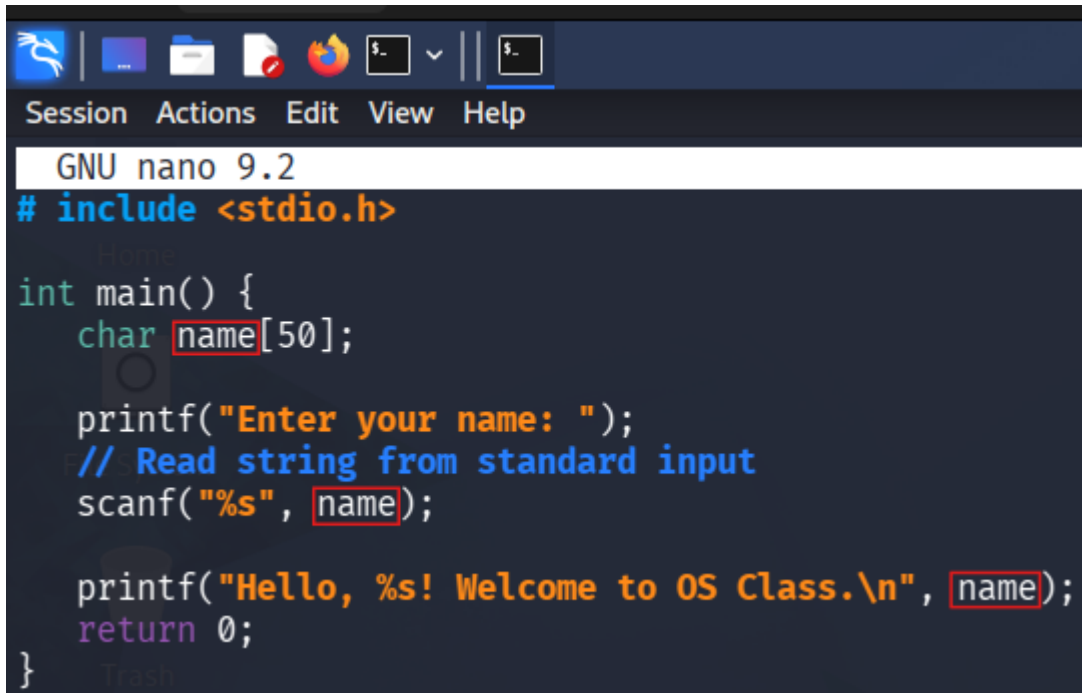
OS Insight: The OS provides standard input (stdin) and standard output (stdout) streams.

scanf reads from **stdin**, and **printf** writes to **stdout**.

A. C Source Code:

A. C Source Code:

C Code program that provides standard input (**stdin**) and standard output (**stdout**) stream.

A screenshot of a terminal window with the nano 9.2 text editor. The code is a C program that prompts the user for their name and prints a welcome message. The code is as follows:

```
GNU nano 9.2
# include <stdio.h>

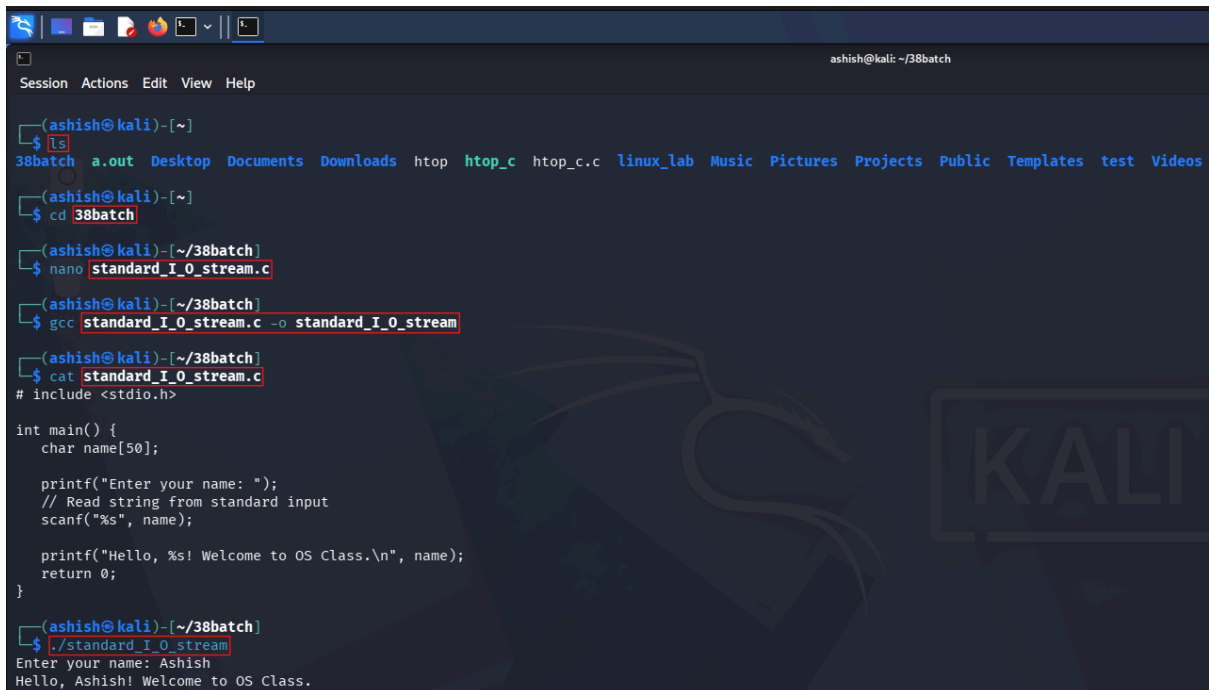
int main() {
    char name[50];

    printf("Enter your name: ");
    // Read string from standard input
    scanf("%s", name);

    printf("Hello, %s! Welcome to OS Class.\n", name);
    return 0;
}
```

Fig17: C source code which provides standard input (stdio) and standard output (stdout).

4. File Operation: This command is used for the OS to provide standard input (stdin) and standard output (stdout) streams. `scanf` reads from stdin and `printf` writes to stdout.

A screenshot of a terminal window showing the execution of a C program. The user is in the ~/38batch directory. They create a file named standard_I_O_stream.c, compile it with gcc, and then run it. The program prompts for a name and prints a welcome message. The terminal output is as follows:

```
ashish@kali: ~/38batch
$ ls
38batch a.out Desktop Documents Downloads htop htop_c htop_c.c linux_lab Music Pictures Projects Public Templates test Videos
$ cd 38batch
$ nano standard_I_O_stream.c
$ gcc standard_I_O_stream.c -o standard_I_O_stream
$ cat standard_I_O_stream.c
# include <stdio.h>

int main() {
    char name[50];

    printf("Enter your name: ");
    // Read string from standard input
    scanf("%s", name);

    printf("Hello, %s! Welcome to OS Class.\n", name);
    return 0;
}
$ ./standard_I_O_stream
Enter your name: Ashish
Hello, Ashish! Welcome to OS Class.
```

Fig18: Command which provides standard input (stdio) and standard output (stdout).

B. Terminal Execution:

➤ **Command to create object for standard_I_O_stream.c:**

`gcc standard_I_O_stream.c -o gcc standard_I_O_stream`

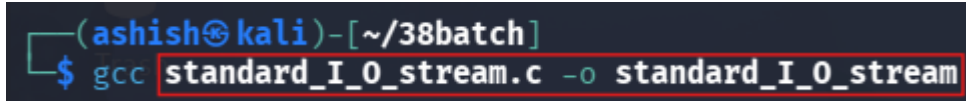
A terminal window with a dark background. The prompt is `(ashish@kali)~[~/38batch]`. The command `$ gcc standard_I_O_stream.c -o standard_I_O_stream` is entered and highlighted with a red box.

Fig19: Command which creates objects for standard_I_O_stream.c file.

➤ **Command to execute code and take user input**

`./standard_I_O_stream.c`

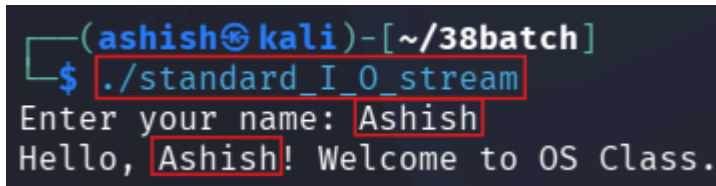
A terminal window with a dark background. The prompt is `(ashish@kali)~[~/38batch]`. The command `$ ./standard_I_O_stream` is entered and highlighted with a red box. Below the command, the text `Enter your name: Ashish` is shown, with `Ashish` highlighted by a red box. The final output is `Hello, Ashish! Welcome to OS Class.`, with `Ashish` highlighted by a red box.

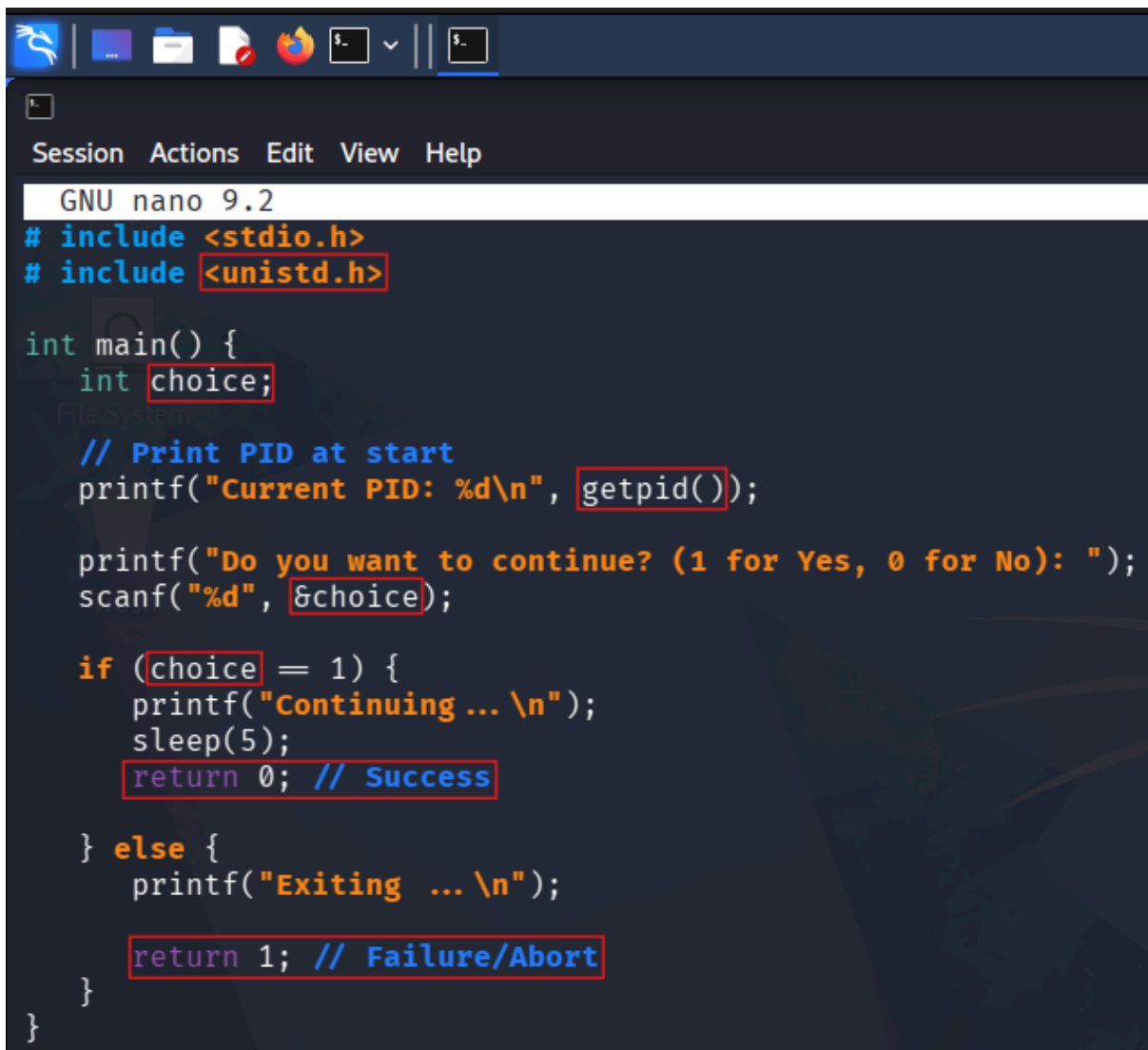
Fig20: Command which takes user input name as string and returns name back to user as output.

VI. TASK 5: CONDITIONAL EXECUTION AND TERMINATION:

OS Insight: Programs often branch based on user input.
The OS tracks the final exit code to determine if the script or
The automation pipeline should continue or halt.

A. C Source Code:

C Code program that return.



```
GNU nano 9.2
# include <stdio.h>
# include <unistd.h>

int main() {
    int choice;

    // Print PID at start
    printf("Current PID: %d\n", getpid());

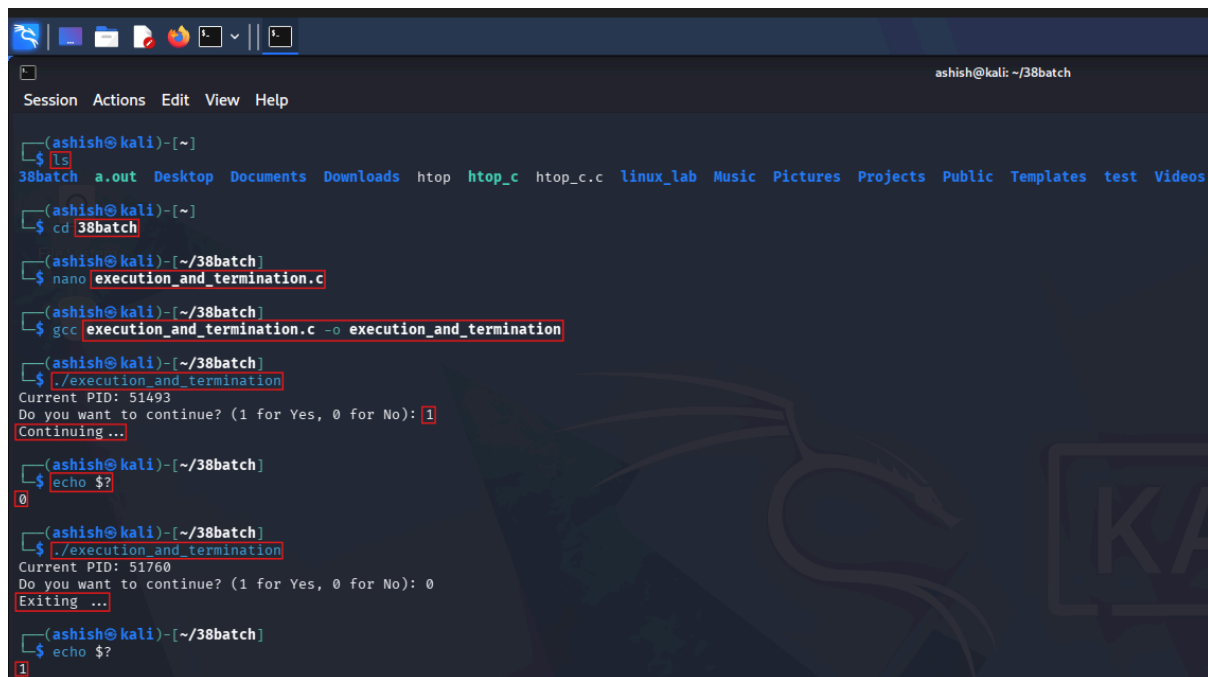
    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing ... \n");
        sleep(5);
        return 0; // Success
    } else {
        printf("Exiting ... \n");

        return 1; // Failure/Abort
    }
}
```

Fig21: C source code which branches on user input and code determines if the scripts or automation pipeline should continue or halt.

5. File Operation: This command is used to branch on user input and code determines if the scripts or automation pipeline should continue or halt.



```
(ashish@kali)-[~]
$ ls
38batch  a.out  Desktop  Documents  Downloads  htop  htop_c  htop_c.c  linux_lab  Music  Pictures  Projects  Public  Templates  test  Videos
(ashish@kali)-[~]
$ cd 38batch
(ashish@kali)-[~/38batch]
$ nano execution_and_termination.c
(ashish@kali)-[~/38batch]
$ gcc execution_and_termination.c -o execution_and_termination
(ashish@kali)-[~/38batch]
$ ./execution_and_termination
Current PID: 51493
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing ...
(ashish@kali)-[~/38batch]
$ echo $?
0
(ashish@kali)-[~/38batch]
$ ./execution_and_termination
Current PID: 51760
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting ...
(ashish@kali)-[~/38batch]
$ echo $?
1
```

Fig22: Command which continues when user input is 1 and exit when user input is 0.

B. Terminal Execution & Verification:

➤ Command to create object for `execution_and_termination.c`:

`gcc execution_and_termination.c -o execution_and_termination`

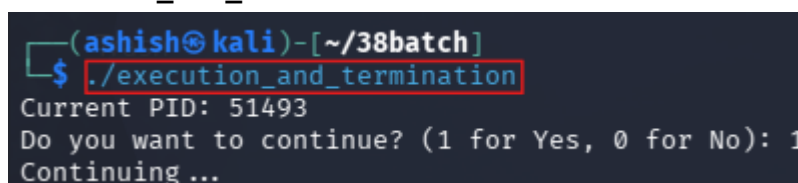


```
(ashish@kali)-[~/38batch]
$ gcc execution_and_termination.c -o execution_and_termination
```

Fig23: Command which creates an object for `execution_and_termination.c`.

➤ Command that continues when user input is 1:

`./execution_and_termination.`

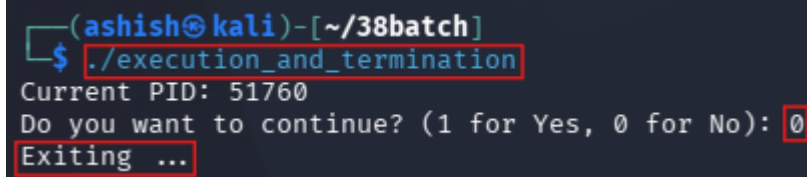


```
(ashish@kali)-[~/38batch]
$ ./execution_and_termination
Current PID: 51493
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing ...
```

Fig24: Command which continues when user input is 1.

➤ **Command that exits when user input is 0:**

`./execution_and_termination.`



```
(ashish@kali) - [~/38batch]
$ ./execution_and_termination
Current PID: 51760
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting ...
```

Fig25: Command which exits when user input is 0.

➤ **Command that give output as 0 using echo \$:**

`./execution_and_termination.`

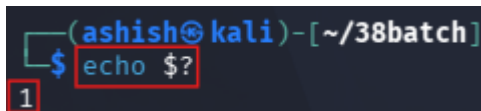


```
(ashish@kali) - [~/38batch]
$ echo $?
0
```

Fig26: `echo $?` Command which exits when user input is 0.

➤ **Command that give output as 1 using echo \$:**

`./execution_and_termination.`



```
(ashish@kali) - [~/38batch]
$ echo $?
1
```

Fig27: `echo $?` Command which continues when user input is 1.

VII. CONCLUSION:

This lab demonstrated that C programs do not exist in a vacuum; they are active processes managed by the Linux kernel. By utilizing system calls like getpid(), sleep(), and return codes, we successfully bridged the gap between high-level C code and low-level OS process management.